

TSRA: Trace-Supervised Reasoning Attention for Multi-step Text Reasoning

Abstract

Multi-step reasoning over text requires models to compose local facts into conclusions, a capability often brittle in systematic generalization settings. We propose **TSRA** (Trace-Supervised Reasoning Attention), a fully differentiable module that instantiates a soft entity graph within a pretrained encoder. TSRA introduces a query-guided step-selection attention head trained with reasoning traces (when available) to explicitly learn compositional inference paths. We further introduce a bijective renaming regularization to enforce consistency. Experiments on CLUTRR and RuleTaker demonstrate that TSRA significantly improves length and depth generalization compared to standard Transformer baselines.

1. Introduction

Multi-step reasoning over text often requires a model to *compose* multiple local facts into a final conclusion. While large pretrained Transformers provide strong pattern recognition (Vaswani et al., 2017; Devlin et al., 2019; Liu et al., 2019), they can be brittle under *systematic generalization* settings where test instances require longer reasoning chains than those seen during training. Two representative benchmarks make this stress test explicit: CLUTRR measures relational reasoning as a function of shortest-path hop length (Sinha et al., 2019), and RuleTaker evaluates rule-based entailment as a function of proof depth (Clark et al., 2020).

A common difficulty is that the intermediate reasoning trajectory is *latent*: models are trained only with the final label, yet are expected to repeatedly decide *where to look next*. In entity-centric problems, this can be viewed as a sequence of discrete “jumps” between entities (or mentions) to gather evidence. If training data provides a reasoning trace (e.g., a shortest path in synthetic relational graphs), it is natural to use it to shape the model’s internal step-selection behavior—but without requiring any trace at inference time.

We propose **TSRA** (Trace-Supervised Reasoning Attention), a simple training strategy that turns this intuition into a fully differentiable module. TSRA instantiates an internal *soft entity graph* inside a pretrained encoder: text is encoded into token representations; entity mentions are

aligned to token spans and pooled into entity vectors; and an entity-attention layer produces a matrix of *step-selection logits* indicating, for each current entity, which entity should be attended to next. Crucially, when traces are available, TSRA applies cross-entropy *directly on these pre-softmax attention logits*, so the supervision acts on the step-selection distribution itself rather than on an auxiliary classifier. At test time, TSRA requires **text only**.

We additionally adopt a lightweight invariance regularizer tailored to compositional generalization. By applying a bijection that renames entity surface forms (semantics preserved), we obtain an augmented view of the same instance and penalize prediction drift between the two views, akin to consistency regularization (Miyato et al., 2018; Xie et al., 2020; Liang et al., 2021). This encourages the model to rely on relational structure rather than memorizing entity identities.

Contributions. This paper makes the following contributions: (i) we formalize TSRA as a trace-supervised step-selection attention mechanism for entity-centric multi-step reasoning; (ii) we show how to attach next-step (and optional edge-relation) supervision directly to attention logits while keeping inference text-only; (iii) we incorporate a simple bijective renaming augmentation with an explicit consistency objective; and (iv) we evaluate under depth-based systematic generalization protocols on CLUTRR (Sinha et al., 2019) and RuleTaker (Clark et al., 2020).

2. Related Work

Multi-step reasoning and systematic generalization. Multi-hop question answering and rule-based reasoning datasets explicitly test whether models can compose evidence across steps, often by controlling reasoning depth. HotpotQA provides strong supervision for supporting facts, motivating architectures that expose intermediate selection decisions (Yang et al., 2018). CLUTRR and RuleTaker further emphasize *length/depth generalization*, where models must extrapolate to longer chains than observed during training (Sinha et al., 2019; Clark et al., 2020).

Neural-symbolic reasoning and differentiable rule learning. Neural-symbolic methods aim to combine distributed representations with the compositionality of symbolic rea-

soning. Representative directions include differentiable theorem proving (Rocktäschel & Riedel, 2017), probabilistic logic programming with neural predicates (Manhaeve et al., 2018), and differentiable rule learning for knowledge base reasoning (Yang et al., 2017; Dong et al., 2019). Compared to explicit symbolic pipelines, TSRA targets a complementary setting: it keeps the input and inference fully in natural language while using traces (when available) only as training-time signals to shape internal step selection.

Intermediate supervision and supervised attention. A long line of work uses auxiliary supervision to guide intermediate decisions, including supervised attention for alignment in neural machine translation (Mi et al., 2016). At the same time, the interpretability of attention has been debated (Jain & Wallace, 2019); our goal is not to treat attention as a faithful explanation, but to use trace supervision as *learning signal* that biases the model toward compositional step selection under distribution shift.

3. Method

3.1. Problem Setting

Given a context/story s and a query q , our goal is to predict a label $y \in \mathcal{Y}$ (e.g., a kinship relation in CLUTRR (Sinha et al., 2019) or entailment in RuleTaker (Clark et al., 2020)). We study *systematic generalization* by evaluating performance as a function of reasoning depth (e.g., hop length in CLUTRR or proof depth in RuleTaker). At inference time, the model takes **text only**. For synthetic datasets that provide additional structure (e.g., a shortest reasoning trace), we use such signals *only during training* to supervise intermediate step decisions.

3.2. Overview

We instantiate a *soft graph* over discrete units extracted from text (entities in our instantiation) inside a pretrained encoder. The model consists of: (i) a Transformer text encoder producing token representations, (ii) a span alignment module that pools token spans into unit vectors, and (iii) a **Trace-Supervised Reasoning Attention (TSRA)** layer that updates unit states and exposes *step-selection logits* for supervision when a reasoning trace is available. TSRA combines two complementary computations:

- a **global aggregation** component that composes evidence across all units to enrich entity representations with context, and
- a **step-selection** component that assigns a next-step preference distribution (as *pre-softmax* attention logits), explicitly guided by the query target.

The final prediction combines a global classifier on the CLS token and a pair classifier on the (updated) query units.

3.3. Text Encoder and Span Pooling

We encode the input pair with a pretrained Transformer f_θ :

$$\mathbf{H} = f_\theta(s, q), \quad \mathbf{H} \in \mathbb{R}^{L \times H}, \quad (1)$$

where L is the input length and H is the hidden size. Let $\mathcal{E} = \{e_1, \dots, e_N\}$ denote the extracted discrete units (entities in our instantiation) and let $\text{span}(i) = [a_i, b_i]$ be the token span aligned to e_i (Section A.1). We obtain a unit embedding by mean pooling:

$$\mathbf{e}_i = \frac{1}{b_i - a_i} \sum_{t=a_i}^{b_i-1} \mathbf{H}_t \in \mathbb{R}^H, \quad \mathbf{E} = [\mathbf{e}_1; \dots; \mathbf{e}_N] \in \mathbb{R}^{N \times H}. \quad (2)$$

We use a boolean validity mask $\mathbf{m} \in \{0, 1\}^N$ with $m_i = 1$ iff $\text{span}(i)$ exists; invalid units are padded and must be masked in all unit-level computations.

The query specifies a subject and an object index (s, o) . We will refer to o as the *query target* (i.e., the unit mentioned in the question whose relation/entailment w.r.t. s is asked). The step-selection component uses this target to make intermediate step choices query-dependent (Section 3.4).

3.4. Trace-Supervised Reasoning Attention

We present a single TSRA layer and omit the batch dimension for clarity.

Pairwise latent type distribution (internal conditioning).

For each pair (i, j) , we predict a distribution over R latent pairwise types. To avoid line overflow, we present the computation of logits and probabilities separately:

$$\ell_{ij} = \text{MLP}_{\text{type}}([\mathbf{e}_i; \mathbf{e}_j; \mathbf{e}_i \odot \mathbf{e}_j]) \in \mathbb{R}^R, \quad (3)$$

$$\mathbf{p}_{ij} = \text{Softmax}(\ell_{ij}) \in \Delta^{R-1}. \quad (4)$$

Conditioning via FiLM on values. Let $\{\mathbf{\Gamma}_r\}_{r=1}^R$ and $\{\mathbf{B}_r\}_{r=1}^R$ be learnable embeddings in $\mathbb{R}^H$. We compute pair-specific FiLM parameters by expectation under $\mathbf{p}_{ij}$:

$$\gamma_{ij} = \sum_{r=1}^R p_{ij,r} \mathbf{\Gamma}_r, \quad \beta_{ij} = \sum_{r=1}^R p_{ij,r} \mathbf{B}_r, \quad \gamma_{ij}, \beta_{ij} \in \mathbb{R}^H. \quad (5)$$

Given a value vector $\mathbf{v}_j \in \mathbb{R}^H$, FiLM yields a pair-conditioned value:

$$\tilde{\mathbf{v}}_{ij} = \mathbf{v}_j \odot (1 + \gamma_{ij}) + \beta_{ij}. \quad (6)$$

Pairwise bias on attention logits. We introduce component-specific scalar biases $\{b_r^g\}_{r=1}^R$ (global) and

$\{b_r^s\}_{r=1}^R$ (step-selection). For each pair, we compute a bias by expectation under $\mathbf{p}_{ij}$:

$$\delta_{ij}^g = \sum_{r=1}^R p_{ij,r} b_r^g, \quad \delta_{ij}^s = \sum_{r=1}^R p_{ij,r} b_r^s. \quad (7)$$

Global aggregation component. This component allows entities to gather context from semantic neighbors regardless of the reasoning path. We apply dot-product attention over units:

$$\mathbf{q}_i^g = \mathbf{W}_Q^g \mathbf{e}_i, \quad \mathbf{k}_j^g = \mathbf{W}_K^g \mathbf{e}_j, \quad \mathbf{v}_j^g = \mathbf{W}_V^g \mathbf{e}_j, \quad (8)$$

and define pre-softmax scores

$$S_{ij}^g = \frac{\langle \mathbf{q}_i^g, \mathbf{k}_j^g \rangle}{\sqrt{H}} + \delta_{ij}^g. \quad (9)$$

After masking invalid units,

$$A_{ij}^g = \text{Softmax}(S_{i,:}^g)_j, \quad \mathbf{m}_i^g = \sum_{j=1}^N A_{ij}^g \tilde{\mathbf{v}}_{ij}^g, \quad (10)$$

where $\tilde{\mathbf{v}}_{ij}^g$ is computed by Equation (6) with $\mathbf{v}_j^g$.

Query-guided step-selection component. Unlike the global component, this head is explicitly conditioned on the target entity o to identify the next reasoning hop. We define:

$$\mathbf{q}_i^s = \mathbf{W}_Q^s \mathbf{e}_i + \mathbf{W}_{qry} \mathbf{e}_o, \quad \mathbf{k}_j^s = \mathbf{W}_K^s \mathbf{e}_j, \quad \mathbf{v}_j^s = \mathbf{W}_V^s \mathbf{e}_j, \quad (11)$$

where $\mathbf{W}_{qry} \in \mathbb{R}^{H \times H}$ is a learnable projection matrix that injects the target goal into the query. We then compute step-selection logits:

$$S_{ij}^s = \frac{\langle \mathbf{q}_i^s, \mathbf{k}_j^s \rangle}{\sqrt{H}} + \delta_{ij}^s, \quad S^s \in \mathbb{R}^{N \times N}. \quad (12)$$

Messages are computed as

$$A_{ij}^s = \text{Softmax}(S_{i,:}^s)_j, \quad \mathbf{m}_i^s = \sum_{j=1}^N A_{ij}^s \tilde{\mathbf{v}}_{ij}^s. \quad (13)$$

When a reasoning trace is available, we supervise S^s directly as next-step logits (Section 3.6).

Fusion and state update. Both messages are fused to update the entity state:

$$\mathbf{e}_i' = \text{LN}(\mathbf{e}_i + \text{Drop}(\mathbf{m}_i^g + \mathbf{m}_i^s)), \quad \mathbf{E}' \in \mathbb{R}^{N \times H}. \quad (14)$$

Algorithm 1 TSRA with Direct Next-Step Supervision

- 1: **Input:** token states $\mathbf{H}$; spans $\{\text{span}(i)\}$; query indices (s, o) ; optional trace $(v_0=s, \dots, v_T=o)$.
 - 2: **Output:** updated unit states $\mathbf{E}'$; step-selection logits S^s .
 - 3: **Span pooling.**
 - 4: Pool unit vectors $\mathbf{E} \in \mathbb{R}^{N \times H}$ and validity mask $\mathbf{m} \in \{0, 1\}^N$.
 - 5: **Pairwise conditioning.**
 - 6: Compute pair logits ℓ_{ij} and probabilities $\mathbf{p}_{ij}$ (Eq. 3).
 - 7: Compute FiLM parameters $(\gamma_{ij}, \beta_{ij})$ (Eq. 5) and bias terms $(\delta_{ij}^g, \delta_{ij}^s)$ (Eq. 7).
 - 8: **Global aggregation.**
 - 9: Compute global scores S^g (Eq. 9) and messages $\mathbf{m}^g$ (Eq. 10).
 - 10: **Step selection.**
 - 11: Compute step-selection logits S^s (Eq. 12) and messages $\mathbf{m}^s$ (Eq. 13).
 - 12: **Update.**
 - 13: $\mathbf{E}' \leftarrow \text{LN}(\mathbf{E} + \text{Drop}(\mathbf{m}^g + \mathbf{m}^s))$ (Eq. 14)
 - 14: **if** trace is provided **then**
 - 15: **Direct next-step supervision.**
 - 16: $\mathcal{L}_{step} \leftarrow \sum_{t=0}^{T-1} \text{CE}(S_{v_t, :}^s, v_{t+1})$
 - 17: **end if**
 - 18: **return** $\mathbf{E}'$ and S^s
-

3.5. Prediction Head

Let $\mathbf{h}_{cls} \in \mathbb{R}^H$ denote the CLS representation from the encoder, and let $\mathbf{e}_s', \mathbf{e}_o'$ be updated subject/object vectors from TSRA. We compute the logits using two separate branches:

$$\mathbf{z}_{cls} = \mathbf{W}_{cls} \mathbf{h}_{cls} + \mathbf{b}_{cls} \in \mathbb{R}^{|\mathcal{Y}|}, \quad (15)$$

$$\mathbf{z}_{pair} = \text{MLP}_{pair}([\mathbf{e}_s'; \mathbf{e}_o'; \mathbf{e}_s' \odot \mathbf{e}_o']) \in \mathbb{R}^{|\mathcal{Y}|}, \quad (16)$$

where $\mathbf{W}_{cls} \in \mathbb{R}^{|\mathcal{Y}| \times H}$ and $\mathbf{b}_{cls} \in \mathbb{R}^{|\mathcal{Y}|}$ are learnable parameters, and $|\mathcal{Y}|$ is the number of target relations. We then fuse them to obtain the final prediction:

$$\mathbf{z} = \mathbf{z}_{cls} + \mathbf{z}_{pair}. \quad (17)$$

3.6. Training Objective

$$\mathcal{L} = \mathcal{L}_{main} + \lambda_{step} \mathcal{L}_{step} + \lambda_{edge} \mathcal{L}_{edge} + \lambda_{cons} \mathcal{L}_{cons}. \quad (18)$$

4. Experiments

We evaluate TSRA on depth-controlled systematic generalization benchmarks: CLUTRR (reasoning hop length) (Sinha et al., 2019) and RuleTaker (proof depth) (Clark et al., 2020). All reported CLUTRR results are mean $\pm$ standard deviation over five independent runs.

4.1. CLUTRR: Hop-Length Generalization

Setup. We compare pretrained Transformer baselines (without TSRA) against the same backbone augmented with TSRA. We report overall accuracy, short-hop accuracy (2–3 hops), and long-hop accuracy (≥ 6 hops), following the hop-based evaluation protocol in CLUTRR (Sinha et al., 2019).

Results. Table 1 shows that adding TSRA consistently improves performance, with the largest gains on long-hop instances. This supports the main motivation of TSRA: using trace supervision to bias the model toward better multi-step step selection, improving length generalization beyond what is achieved by end-task supervision alone.

4.2. RuleTaker: Depth Generalization

We additionally report depth-specific accuracy on RuleTaker (Clark et al., 2020). For this experiment we report the best accuracy observed over 20 epochs (with data augmentation enabled).

Table 1. Reasoning Performance on CLUTRR (Reasoning Hop) Dataset

Model	Overall Acc	Short Hop (2-3)	Long Hop (≥ 6)
1. Baseline Pre-trained Transformer Models (No TSRA)			
BERT-Base	0.195 ± 0.031	0.682 ± 0.026	0.106 ± 0.040
RoBERTa-Base	0.345 ± 0.027	0.759 ± 0.020	0.227 ± 0.034
DeBERTa-Base	0.347 ± 0.024	0.764 ± 0.018	0.232 ± 0.031
DeBERTa-V3-Base	0.397 ± 0.021	0.783 ± 0.016	0.264 ± 0.028
ModernBERT-Base	0.345 ± 0.023	0.777 ± 0.017	0.227 ± 0.029
2. Improved Models with Trace-Supervised Reasoning Attention (TSRA)			
BERT-Base + TSRA	0.496 ± 0.029	0.728 ± 0.022	0.372 ± 0.035
ModernBERT-Base + TSRA	0.588 ± 0.019	0.805 ± 0.014	0.427 ± 0.025
DeBERTa-V3-Base + TSRA	0.615 ± 0.018	0.790 ± 0.013	0.486 ± 0.024
RoBERTa-Base + TSRA	0.6353 ± 0.013	0.7825 ± 0.010	0.4652 ± 0.019
DeBERTa-Base + TSRA	0.6719 ± 0.015	0.8117 ± 0.012	0.4840 ± 0.021

Note: All values are **mean $\pm$ standard deviation** from five independent experiments; bold values denote the best performance for each metric.

CLUTRR: Short Hop (2-3) = 2–3 hops reasoning accuracy, Long Hop (≥ 6) = at least 6 hops reasoning accuracy; higher values indicate better performance.

Table 2. Depth Reasoning Performance on RuleTaker Dataset (TSRA Models Only)

Model	depth-0	depth-1	depth-2	depth-3	depth-5	Macro Avg
RoBERTa-Base + TSRA	0.9923	0.9907	0.9853	0.9666	0.9007	0.9641

Note: Values are the **global optimal accuracy** from 20 training epochs of the TSRA model with data augmentation enabled.

A. Appendix

A.1. Entity Span Alignment and Masking

We align surface forms to token spans to pool unit vectors from token states. Given a raw text and a name string, we first attempt an *exact bracket match* (e.g., [Alice]) and fall back to a raw string match if needed. We use tokenizer offset mappings to convert character spans to token spans. If a name appears multiple times, we choose the first occurrence for stability. If alignment fails, we set the span to invalid (padded as $(-1, -1)$ in the batch tensor), and its pooled embedding is set to the zero vector.

Masking. Invalid units must be masked in (i) attention logits and (ii) any auxiliary supervision that indexes unit nodes. In our implementation, padded logits are filled with a large negative constant (e.g., -10^4), so that Softmax assigns negligible probability mass and cross-entropy remains well-defined.

A.2. Graph Construction and Shortest-Trace Labels in CLUTRR

For CLUTRR, training data includes a kinship graph specified by edges and relation types. We construct a directed graph by adding: (i) the original directed edges with their relation type, and (ii) an inverse edge for each original edge. The inverse relation label is inferred from a predefined mapping (e.g., *father* $\leftrightarrow$ *son/daughter*) using the gender of the inverse edge’s source node; if no mapping exists we fall back to an *inverse_prefix*.

We then compute the shortest directed trace from the subject to the object unit and record: (i) the node index sequence $(v_0, \dots, v_T)$, and (ii) the edge label sequence $(r_0, \dots, r_{T-1})$ along the trace. These labels are used only for training auxiliary losses.

A.3. Model Details and Complexity

Per-sample computation for variable N . TSRA requires pairwise interactions and scales as $O(N^2)$ per sample. To avoid quadratic overhead from padding to the maximum N in a batch, we compute attention per sample on its valid prefix and write the resulting step-selection logits into a padded tensor initialized with a large negative constant.

A.4. Full Loss Definitions

We minimize a joint objective $\mathcal{L}$ comprising four terms. Below we provide precise definitions for each term.

1. Main Task Loss ($\mathcal{L}_{main}$). Let $\mathbf{z} \in \mathbb{R}^{|\mathcal{Y}|}$ be the predicted logits for the original input sample, and $y \in \{1, \dots, |\mathcal{Y}|\}$ be the ground-truth relation label index. When data augmentation (bijective renaming) is enabled, let $\tilde{\mathbf{z}}$ denote the logits predicted from the renamed view of the same sample. We compute the Cross-Entropy (CE) loss averaged over both views:

$$\mathcal{L}_{main} = \begin{cases} \text{CE}(\mathbf{z}, y), & \text{if augmentation is disabled,} \\ \frac{1}{2}\text{CE}(\mathbf{z}, y) + \frac{1}{2}\text{CE}(\tilde{\mathbf{z}}, y), & \text{otherwise.} \end{cases} \quad (19)$$

2. Step-Selection Supervision ($\mathcal{L}_{step}$). This loss supervises the reasoning trajectory. Let $\mathcal{B}_p$ be the set of indices of samples in the current batch that possess a valid ground-truth trace. For a sample $b \in \mathcal{B}_p$, let the trace be a sequence of node indices $(v_0, v_1, \dots, v_{T_b})$, where T_b is the path length (number of hops). We maximize the probability of selecting the correct next node v_{t+1} given the current node v_t using the step-selection logits $S^{s,(b)}$:

$$\mathcal{L}_{step} = \frac{1}{|\mathcal{B}_p|} \sum_{b \in \mathcal{B}_p} \frac{1}{T_b} \sum_{t=0}^{T_b-1} \text{CE}(S_{v_t, :}^{s,(b)}, v_{t+1}), \quad (20)$$

where $S_{v_t, :}^{s,(b)} \in \mathbb{R}^N$ is the row vector of attention logits for the query node v_t in sample b .

3. Edge-Relation Supervision ($\mathcal{L}_{edge}$). This auxiliary loss supervises the intermediate relations along the path. Let $\mathcal{B}_e$ denote the set of all valid edge transitions across the entire batch. Each element in $\mathcal{B}_e$ is a tuple (b, t) indicating the t -th step in sample b . Let $r_t^{(b)}$ be the ground-truth relation label for the edge connecting v_t to v_{t+1} in sample b . We supervise the

pairwise latent type predictor:

$$\mathcal{L}_{edge} = \frac{1}{|\mathcal{B}_e|} \sum_{(b,t) \in \mathcal{B}_e} \text{CE}(\mathbf{h}_{edge}^{(b,t)}, r_t^{(b)}), \quad (21)$$

where the edge representation $\mathbf{h}_{edge}^{(b,t)}$ is computed from the updated unit vectors $\mathbf{E}'^{(b)}$:

$$\mathbf{h}_{edge}^{(b,t)} = \text{MLP}_{type} \left(\left[\mathbf{e}_{v_t}^{(b)}; \mathbf{e}_{v_{t+1}}^{(b)}; \mathbf{e}_{v_t}^{\prime(b)} \odot \mathbf{e}_{v_{t+1}}^{\prime(b)} \right] \right). \quad (22)$$

4. Consistency Regularization ($\mathcal{L}_{cons}$). To encourage the model to focus on structural semantics rather than memorizing surface names, we penalize the divergence between the probability distributions of the original view ($\mathbf{z}$) and the renamed view ($\tilde{\mathbf{z}}$). We use the Kullback-Leibler (KL) divergence:

$$\mathcal{L}_{cons} = D_{\text{KL}}(\text{Softmax}(\tilde{\mathbf{z}}) \parallel \text{Softmax}(\mathbf{z})). \quad (23)$$

Note that $\tilde{\mathbf{z}}$ is detached from the computation graph of $\mathbf{z}$ to avoid collapsing solutions, serving purely as a consistency target.

References

- Clark, P., Tafjord, O., and Richardson, K. Transformers as soft reasoners over language. In *Proceedings of the Twenty-Ninth International Joint Conference on Artificial Intelligence (IJCAI)*, 2020.
- Devlin, J., Chang, M.-W., Lee, K., and Toutanova, K. BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT)*, 2019.
- Dong, H., Mao, J., Lin, T., Wang, C., Li, L., and Blei, D. M. Neural logic machines. In *International Conference on Learning Representations (ICLR)*, 2019.
- Jain, S. and Wallace, B. C. Attention is not explanation. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT)*, 2019.
- Liang, X., Wu, L., Li, J., Wang, Y., Meng, Q., Qin, T., Chen, W., Zhang, M., and Liu, T.-Y. R-Drop: Regularized dropout for neural networks. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2021.
- Liu, Y., Ott, M., Goyal, N., Du, J., Joshi, M., Chen, D., Levy, O., Lewis, M., Zettlemoyer, L., and Stoyanov, V. RoBERTa: A robustly optimized BERT pretraining approach, 2019.
- Manhaeve, R., Dumančić, S., Kimmig, A., Demeester, T., and Raedt, L. D. Deepproblog: Neural probabilistic logic programming. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2018.
- Mi, H., Wang, Z., and Ittycheriah, A. Supervised attentions for neural machine translation. In *Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2016.
- Miyato, T., Ichi Maeda, S., Koyama, M., and Ishii, S. Virtual adversarial training: A regularization method for supervised and semi-supervised learning. *IEEE Transactions on Pattern Analysis and Machine Intelligence (TPAMI)*, 2018.
- Rocktäschel, T. and Riedel, S. End-to-end differentiable proving. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
- Sinha, K., Sodhani, S., Dong, J., Pineau, J., and Hamilton, W. L. CLUTRR: A diagnostic benchmark for inductive reasoning from text. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, 2019.
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., and Polosukhin, I. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
- Xie, Q., Dai, Z., Hovy, E., Luong, M.-T., and Le, Q. V. Unsupervised data augmentation for consistency training. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

- Yang, F., Yang, Z., and Cohen, W. W. Differentiable learning of logical rules for knowledge base reasoning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
- Yang, Z., Qi, P., Zhang, S., Bengio, Y., Cohen, W. W., Salakhutdinov, R., and Manning, C. D. HotpotQA: A dataset for diverse, explainable multi-hop question answering. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2018.